

Lab 7: Event-Driven Architecture (EDA) & Decoupling

System: BadNet - Badminton Social Network **Pattern:** Event-Driven Architecture (RabbitMQ) **Event:** ChallengeCreatedEvent

Introduction

This lab implements **Event-Driven Architecture (EDA)** to achieve the **Fault Isolation ASR**. When a player creates a Challenge, the Match Service (Producer) publishes an event to the message broker instead of calling the Notification Service directly. The Notification Service (Consumer) will handle sending notifications asynchronously. This ensures the main flow is not blocked if the notification system is slow or unavailable.

Objectives

1. Understand the concepts of Producers, Consumers, and Message Brokers.
2. Set up RabbitMQ locally.
3. Implement an Order Service (Producer) that publishes events.
4. Implement a Notification Service (Consumer) that processes events.
5. Demonstrate the decoupled and resilient nature of the system.

Technology Stack

Tool	Purpose	Installation
RabbitMQ	Message Broker	Docker
Python + Pika	RabbitMQ Client	<code>pip install pika</code>
Match Service	Event Producer	Port 8001
Notification Service	Event Consumer	Separate process

Activity Practice 1: Setup RabbitMQ & Project

1. Start RabbitMQ using Docker:

Bash

```
docker run -d --hostname rabbitmq-host --name rabbitmq \
  -p 5672:5672 -p 15672:15672 rabbitmq:3-management
```

Access RabbitMQ Management UI: <http://localhost:15672> (Username: guest, Password: guest)

2. Create Project:

Bash

```
mkdir badnet_eda
```

```
cd badnet_eda
```

```
python -m venv venv
```

```
source venv/bin/activate
```

```
pip install pika
```

Activity Practice 2: Match Service (Event Producer)

File: challenge_producer.py

Python

```
import pika
```

```
import json
```

```
import time
```

```
RABBITMQ_HOST = 'localhost'
```

```
QUEUE_NAME = 'badnet_challenge_events'
```

```
def publish_challenge_created_event(challenge_data):
```

```
    try:
```

```
        connection = pika.BlockingConnection(pika.ConnectionParameters(host=RABBITMQ_HOST))
```

```
        channel = connection.channel()
```

```
        channel.queue_declare(queue=QUEUE_NAME)
```

```
        message = json.dumps(challenge_data)
```

```
        channel.basic_publish(
```

```
            exchange="",
```

```
            routing_key=QUEUE_NAME,
```

```
            body=message
```

```

    )

    print(f"[x] Match Service published ChallengeCreatedEvent | ID:
{challenge_data['challenge_id']}")

    connection.close()

except Exception as e:

    print(f"[!] Connection Error: {e}")


if __name__ == '__main__':

    print("BadNet Match Service (Producer) is starting...")

    for i in range(1, 4):

        challenge_info = {

            "challenge_id": f"CHL-{i:03}",

            "challenger_id": i,

            "opponent_id": i + 1,

            "match_date": "2026-06-05 15:00:00",

            "location": "District 7 Badminton Court",

            "timestamp": time.time()

        }

        publish_challenge_created_event(challenge_info)

        time.sleep(1.5)

```

Activity Practice 3: Notification Service (Event Consumer)

File: notification_consumer.py

Python

import pika

import json

import time

RABBITMQ_HOST = 'localhost'

QUEUE_NAME = 'badnet_challenge_events'

```

def callback(ch, method, properties, body):
    try:
        event_data = json.loads(body)
        challenge_id = event_data.get('challenge_id')
        challenger = event_data.get('challenger_id')
        opponent = event_data.get('opponent_id')

        print(f' [x] Notification Service received ChallengeCreatedEvent: {challenge_id}')

        # Simulate sending notification (email / push notification)
        time.sleep(2.5)

        print(f' [✓] SENT NOTIFICATION: Challenge {challenge_id} from player {challenger} to
player {opponent}')

        ch.basic_ack(delivery_tag=method.delivery_tag)

    except Exception as e:
        print(f' [!] Error processing message: {e}')
        ch.basic_reject(delivery_tag=method.delivery_tag, requeue=False)

if __name__ == '__main__':
    try:
        print("BadNet Notification Service (Consumer) is starting...")
        connection = pika.BlockingConnection(pika.ConnectionParameters(host=RABBITMQ_HOST))
        channel = connection.channel()
        channel.queue_declare(queue=QUEUE_NAME)
        channel.basic_qos(prefetch_count=1)

        channel.basic_consume(

```

```

    queue=QUEUE_NAME,
    on_message_callback=callback,
    auto_ack=False
)

print(' [*] Waiting for ChallengeCreatedEvents. Press CTRL+C to exit')
channel.start_consuming()

except KeyboardInterrupt:
    print('Consumer shutting down...')
except Exception as e:
    print(f' [*] Connection Error: {e}')

```

Activity Practice 4: Testing Asynchronous Decoupling

Instructions:

1. Terminal 1 (Consumer - run first):

Bash

```
python notification_consumer.py
```

2. Terminal 2 (Producer):

Bash

```
python challenge_producer.py
```

Expected Behavior:

- The Producer publishes events very quickly.
- The Consumer processes events with a delay (simulating notification sending).
- This proves **temporal decoupling** and **fault isolation** — the Match Service is not blocked by notification processing.